▲ orlp on April 20, 2022 | parent | context | favorite | on: Changing std:sort at Google’s scale and beyond

Hi all, I'm the author of pdqsort that's credited in the post (to be clear: the authors acknowledged pdqsort at the bottom, I am *not* associated with the posted work directly). I recently held a talk at my institute on efficient in-memory sorting and the ideas in pdqsort, in case you're interested in hearing some of the theory behind it all: https://www.youtube.com/watch?v=jz-PBiWwNjc

Next week I will hold another talk in the Dutch seminar on data systems design (https://dsdsd.da.cwi.nl/) on glidesort, a new *stable* sorting algorithm I've been working on. It is a combination of adaptive quicksort (like pdqsort, fully adaptive for many equal elements) and an adaptive mergesort (like timsort, fully adaptive for long pre-sorted runs). It is the first practical implementation of an algorithm I'm aware of that's fully adaptive for both. Like pdqsort it uses modern architecture aware branchless sorting, and it can use an arbitrary buffer size, becoming faster as you give it more memory (although if given a constant buffer size it will degrade to O(n (log n)^2) in theory, in practice for realistic workloads it's just a near-constant factor (c ~<= 3-5) slower).

The source code isn't publish-ready yet, I have to still do some extra correctness vetting and testing, and in particular exception-safety is still not yet fully implemented. This is important because I wrote it in Rust where we must always give back a valid initialized array, even if a comparison operator caused a panic. But I do have some performance numbers to quote, that won't significantly change.

For sorting 2^24 randomly shuffled distinct u32s using a buffer of 2^23 elements (n/2), glidesort beats Rust's stdlib slice::sort (which is a classic timsort also using a buffer of n/2) by a factor of 3.5 times. When stably sorting the same numbers comparing only their least significant 4 bits, it beats stdlib slice::sort by 12.5 times using 6.5 times fewer comparisons, both numbers on my Apple M1 Macbook. All of this is just using single-threaded code with a generic comparison function. No SIMD, no threads, no type-specific optimizations.

Finally, glidesort with a buffer size of >= n/2 is *faster* than pdqsort.

▲ mlochbaum on April 20, 2022 | next [–]

Any chance you could comment on fluxsort[0], another fast quicksort? It's stable and uses a buffer about the size of the original array, which sounds like it puts it in a similar category as glidesort. Benchmarks against pdqsort at the end of that README; I can verify that it's faster on random data by 30% or so, and the stable partitioning should mean it's at least as adaptive (but the current implementation uses an initial analysis pass followed by adaptive mergesort rather than optimistic insertion sort to deal with nearly-sorted data, which IMO is fragile). There's an in-place effort called crumsort[1] along similar lines, but it's not stable.

I've been doing a lot of work on sorting[2], in particular working to hybridize various approaches better. Very much looking forward to seeing how glidesort works.

[0] https://github.com/scandum/fluxsort

[1] https://github.com/scandum/crumsort

[2] https://mlochbaum.github.io/BQN/implementation/primitive/sor...

▲ orlp on April 20, 2022 | parent | next [–]

I have stolen a lot of ideas from scandum and extended his ideas in new ways. He is definitely a mad genius. Glidesort (on my M1 machine at least) matches fluxsort within a couple % for random data, but glidesort is robust in that it will always take advantage of pre-sorted runs and many equal elements (at least if it has buffer memory), no matter where they are in the array.

In particular, I was inspired by three things from scandum:

1. fluxsort's out-of-place stable partitioning. From this I got reminded that not only is out-of-place stable partitioning a thing, it's highly competitive. I've always had this as an idea in the back of my mind, but never went through with it because I kept getting discouraged by C++'s distinction of moving to uninitialized memory vs. moving into a moved-from value (which is why I implemented glidesort in Rust).

2. quadsort's "ping pong merge", which reduces unnecessary memcpys by merging both on the way *out* and on the way *in* the original array. I did have this idea before, but always dismissed it because I thought keeping track of what's where would be a massive pain. Simply waiting until there's 4 things to merge eliminates this problem and is just genius.

3. quadsort's "branchless parity merge", which merges from both ends of the array if the merge is perfectly balanced. I make no claim that I thought of this, it's just genius. I had two key takeaways from this: you can make some very fast small sorting algorithms with merges, and interleaving loops to reduce data dependencies are significantly faster.

So I combined #1 & #3 into what I call bidirectional stable partitioning, where I partition from *both* sides of the array into an out-of-place buffer through interleaved loops.

I extended the adaptiveness and applicability of #2 heavily by replacing the 'merge' operation in powersort (https://arxiv.org/abs/1805.04154) with a 'virtual merge' operation that delays merges until necessary. This is also what allows me to use quicksort in a bottom-up adaptive mergesort, because I don't eagerly sort small runs! Instead I simply keep unsorted runs around, 'merging' unsorted runs simply by concatenating them - purely in bookkeeping.

I heavily extended 3 for the mergesort part by realizing we don't need perfectly balanced merges, we can just take the `min` of the two runs and start off with a merge from both sides, and then look further. I also did more interleaving by doing a binary search to compute independent parallel merges where sensible, and interleaving those loops.

As a quick preview, here is a visualization of glidesort using a buffer size of n/2, where I have artificially limited the concatenation of unsorted runs to n/8 so that it won't just look only like quicksort, and both the quicksort and mergesort aspects are shown: https://cdn.discordapp.com/attachments/273539705595756544/96...

▲ mlochbaum on April 20, 2022 | root | parent | next [–]

Thanks for the discussion! Can't say I follow everything, but using parity merge for part of an unbalanced merge makes a lot of sense and that alone is worth it.

Stepped through the video a few times at 1/4 speed. The n/8 thing is a bit confusing, first because I didn't read it and second because it makes it hard to tell a partition result from the beginning of the next segment. I think I can follow what's going on, but I don't get the purpose of the bidirectional partition. It doesn't use less memory, does it? So is there something to do with fitting in with mergesort better? I'm not familiar with powersort; I'll read up on it.

▲ orlp on April 20, 2022 | root | parent | next [–]

> but I don't get the purpose of the bidirectional partition. It doesn't use less memory, does it? So is there something to do with fitting in with mergesort better

Nope, it does the same amount of comparisons, same number of operations, same memory, etc. What it does do is it allows you to interleave two independent loops, which is also what makes the parity merge fast (I think scandum misidentifies the loop unrolling for this effect for large arrays - you can loop unroll merging large arrays either way - for small constant-size merging it is important however).

A modern CPU has a very long pipeline, and even though we like to pretend all instructions are one cycle with no latency, in reality there are real latencies to instructions and memory accesses, and multiple instructions can be executed in the same cycle. Since each iteration of the partition depends on the previous one (which pointer did we increment? we have to know before we can execute the next store), you can hide these latencies better if you interleave two independent loops. In addition you can use more instruction-level parallelism.

▲ mlochbaum on April 20, 2022 | root | parent | next [–]

Got it. I'd considered that but something about your explanation threw me off. A subtlety I missed was that you can't just do two forward partitions, because the second one begins exactly halfway through the array—one of the results would be placed there but it's probably not the correct start location for the larger half-partition.

▲ orlp on April 20, 2022 | root | parent | next [–]

Exactly, which is why I call it bidirectional partitioning: one forward, one backward. It's a very strange case where you can use parallelism (in this case instruction-level parallelism), but only get two independent instances without the ability to recurse further.

You can of course make partitioning embarrassingly parallel, look at IPS4o for that. But it is vastly more complicated, and involves overhead shuffling blocks after the partition.

▲ jiggawatts on April 21, 2022 | root | parent | prev | next [–]

I got that impression from your linked video -- the algorithm looks cache-friendly *and* pipeline friendly.

▲ johnx123-up on April 21, 2022 | parent | prev | next [–]

FWIW, I have found scandum's note on pdqsort in HN https://news.ycombinator.com/threads?id=scandum

▲ hengri on April 20, 2022 | prev | next [–]

Random question, but is this from the Google bare metal team? I remember interviewing with them and the work they did was fascinating.

▲ danlark on April 20, 2022 | parent | next [–]

Hi, the author is here

I am working in MapReduce/Data-pipelining/Dataproc efficiency internally and externally. In cloud you can check out Google Cloud Dataflow

We are working closely with general efficiency and bare metal teams, yes :). They definitely do a fascinating work. This one was my 20% project together with the google efficiency team

▲ orlp on April 20, 2022 | parent | prev | next [–]

I'm not sure if you're asking me personally or about the original post, but to be clear: I'm not (currently) associated with Google, nor is my work on pdqsort or glidesort. I'm also not associated with the original post, other than that my work on pdqsort was credited at the bottom of the blog post, which is what my opening sentence was referring to.

I edited my comment to reflect this, I can see how it could've been misinterpreted. It was not my intention to deceive.

▲ omegalulw on April 21, 2022 | prev | next [–]

Is the buffer here extra memory in addition to the input? What about just an O(log N) size buffer?

▲ orlp on April 21, 2022 | parent | next [–]

> Is the buffer here extra memory in addition to the input?

Yes.

> What about just an O(log N) size buffer?

That would effectively just be a constant sized buffer. Honestly O(log N) factors where N is the size of the input are overrated and are probably best seen as a constant factor. Like, even if you're sorting 16 gigabytes of 32-bit floats, log2 N is still just 32.

▲ omegalulw on April 21, 2022 | root | parent | next [–]

Thanks. As a follow up, isn't regular quicksort just using O(log N) memory - for the recursion stack? Assuming random pivot and considering average performance. Seems like O(log N) is worth comparing to...?

▲ orlp on April 21, 2022 | root | parent | next [–]

Glidesort also - from an academic standpoint - always uses O(log N) memory, log2(N)*4 words to be precise.

But on 64-bit machines log2(N) is always <= 64, so I just allocate a 64-element array of 4 word structs on the stack.

I think it's just much more productive and predictive to say that "glidesort has performance X when using a 512-element buffer" than "glidesort has performance X when using a c log (n) buffer".

▲ vanderZwan on April 21, 2022 | prev | next [–]

Will you take on radix sorting algorithms next? :)

▲ a1369209993 on April 21, 2022 | prev [–]

> https://www.youtube.com/watch?v=jz-PBiWwNjc

FWIW, the 360p mp4 (youtube-dl -f18) encoding of this seems to be truncated ("Got server HTTP error: Downloaded 989757 bytes, expected 61794145 bytes."), so you might want to try and get youtube to reencode it. (-f22/720p works fine, so it's probably a transient error rather than a problem with your original upload.)